

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
«Запросы на выборку и модификацию данных. Представления.
Работа с индексами»

По дисциплине «Проектирование и реализация баз данных»

Вариант 11

Обучающийся: Востров Илья Анатольевич

Факультет: ФПИИ

Группа: К3239

Направление подготовки: 09.03.03 Прикладная информатика

Образовательная программа: Мобильные и сетевые технологии

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
Цель работы	3
Практическое задание	3
Логическая модель базы данных (ERD)	4
Выполнение	5
Запросы к базе данных на выборку	5
Запрос 1: Поиск специалиста по конкретной марке автомобиля.	5
Запрос 2: Выявление постоянных клиентов механика.	6
Запрос 3: Контроль соблюдения сроков выполнения заказов.	7
Запрос 4: Поиск самого частого клиента за прошлый год.	8
Запрос 5: Расчет выручки мастеров за месяц.	9
Запрос 6: Поиск вернувшихся клиентов.	10
Запрос 7: Расчет финансовых штрафов для механиков.	11
Создание виртуальных таблиц	12
Представление 1 (Для заказчиков): Статистика специализации мастеров.	12
Представление 2 (Для менеджеров): Автоматический расчет премий.	13
Модификация данных с использованием вложенных подзапросов	14
Команда UPDATE.	14
Команда INSERT.	17
Команда DELETE.	20
Индексы и анализ планов запросов	23
Простой индекс.	23
Составной индекс	27
ВЫВОДЫ	29

Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) **с использованием подзапросов**.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Логическая модель базы данных (ERD)

Схема инфологической модели данных представлена на рисунке 1:

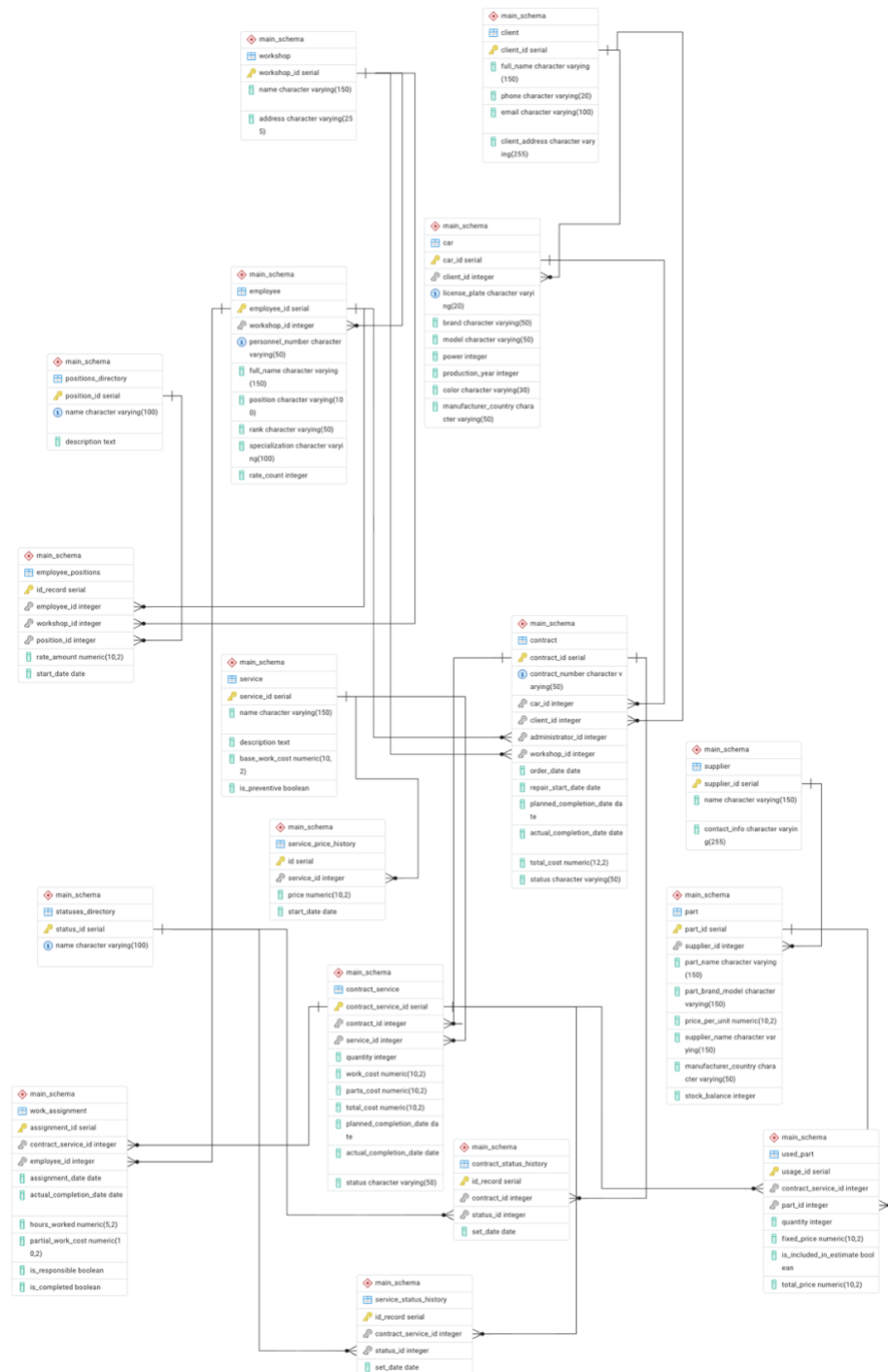


Рисунок 1 — Логическая модель базы данных (ERD)

Выполнение

Запросы к базе данных на выборку

Запрос 1: Поиск специалиста по конкретной марке автомобиля.

Цель: Найти фамилию механика, который чаще всех ремонтирует автомобили марки «Toyota».

Что я сделал: Сформировал запрос с объединением таблиц сотрудников, распределения работ, договоров и автомобилей. Произвел фильтрацию по марке авто. Данные сгруппировал по ФИО механиков с подсчетом количества ремонтов, после чего отсортировал по убыванию с ограничением вывода одной строки.

Запрос

История запросов

1

SET search_path TO main_schema;

2

3

SELECT employee.full_name AS "Механик", COUNT(car.car_id) AS "Ремонтов Тойоты"

4

FROM employee

5

JOIN work_assignment ON employee.employee_id = work_assignment.employee_id

6

JOIN contract_service ON work_assignment.contract_service_id = contract_service.contract_service_id

7

JOIN contract ON contract_service.contract_id = contract.contract_id

8

JOIN car ON contract.car_id = car.car_id

9

WHERE car.brand = 'Toyota'

10

GROUP BY employee.full_name

11

ORDER BY "Ремонтов Тойоты" DESC

12

LIMIT 1;

13

14

Результат

Сообщения

Уведомления

Showing rows: 1 to 1

Page No: 1

of 1

	Механик character varying (150)	Ремонтов Тойоты bigint
1	Тойотов Т.Т.	2

Рисунок 2 — Результаты Запроса 1

Запрос 2: Выявление постоянных клиентов механика.

Цель: Определить клиентов, которые обращались в сервис более одного раза, но при этом их всегда обслуживал один и тот же мастер.

Что я сделал: Связал таблицы клиентов и механиков. Использовал оператор HAVING для наложения условия на сгруппированные данные, где количество уникальных механиков для клиента строго равно 1, а общее количество договоров строго больше 1.

The screenshot shows a database query interface. The top section is titled "Запрос" (Query) and "История запросов" (Query History). The SQL query is displayed in a text area with line numbers 13 to 26. The query is as follows:

```
13
14
15
16 SELECT client.full_name AS "Клиент", employee.full_name AS "Постоянный мастер"
17 FROM client
18 JOIN contract ON client.client_id = contract.client_id
19 JOIN contract_service ON contract.contract_id = contract_service.contract_id
20 JOIN work_assignment ON contract_service.contract_service_id = work_assignment.contract_service_id
21 JOIN employee ON work_assignment.employee_id = employee.employee_id
22 GROUP BY client.client_id, client.full_name, employee.full_name
23 HAVING COUNT(DISTINCT employee.employee_id) = 1 AND COUNT(contract.contract_id) > 1;
24
25
26
```

Below the query is a toolbar with icons for various actions. The bottom section is titled "Результат" (Result) and "Сообщения" (Messages). It shows the results of the query in a table with two columns: "Клиент" (Client) and "Постоянный мастер" (Permanent Master). The table has two rows of data.

	Клиент character varying (150)	Постоянный мастер character varying (150)
1	Алексеев А.А.	Тойотов Т.Т.
2	Борисов Б.Б.	Универсалов У.У.

Рисунок 3 — Результаты Запроса 2

Запрос 3: Контроль соблюдения сроков выполнения заказов.

Цель: Вывести список механиков, нарушивших сроки ремонта, с подсчетом суммарного количества дней просрочки.

Что я сделал: Отфильтровал договоры, в которых фактическая дата завершения превышает плановую. Разница между датами просуммировал и сгруппировал по сотрудникам.

The screenshot shows a database query interface. The top tab is 'Запрос' (Query) with a sub-tab 'История запросов' (Query History). The SQL query is as follows:

```
25
26
27 SELECT employee.full_name AS "Механик",
28        SUM(contract.actual_completion_date - contract.planned_completion_date) AS "Дней просрочки"
29 FROM employee
30 JOIN work_assignment ON employee.employee_id = work_assignment.employee_id
31 JOIN contract_service ON work_assignment.contract_service_id = contract_service.contract_service_id
32 JOIN contract ON contract_service.contract_id = contract.contract_id
33 WHERE contract.actual_completion_date > contract.planned_completion_date
34 GROUP BY employee.full_name;
35
36
37
```

The bottom tab is 'Результат' (Result). It shows a table with two columns: 'Механик' (Mechanic) and 'Дней просрочки' (Days overdue). The data is as follows:

Механик	Дней просрочки
Косяков К.К.	3

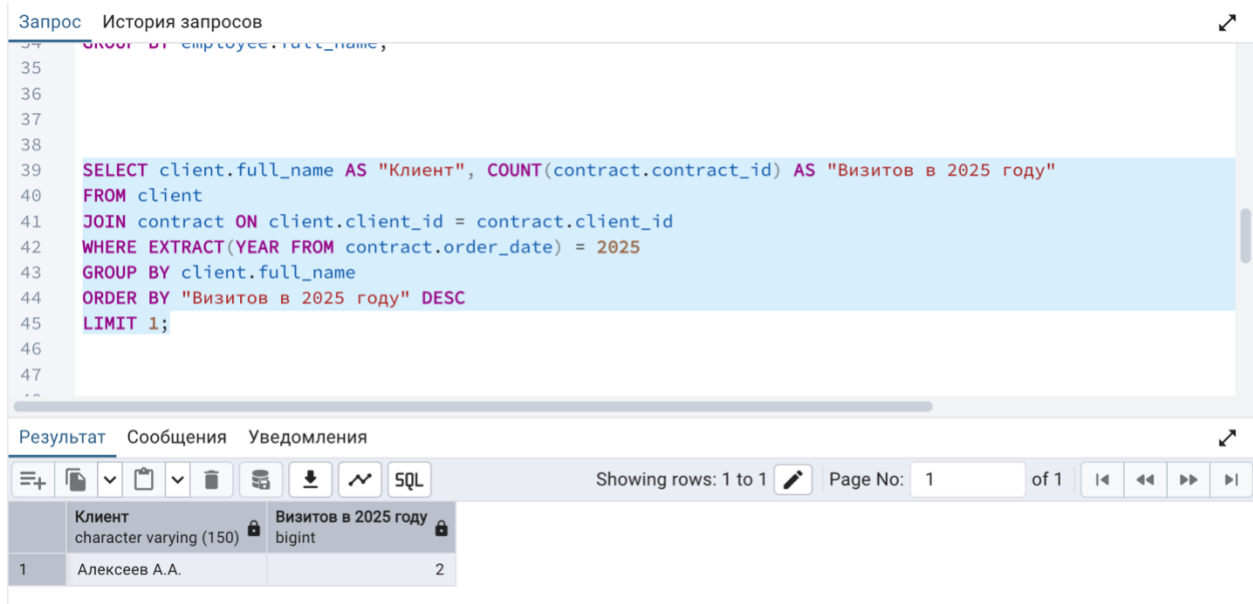
The interface also includes a toolbar with icons for various actions and a status bar indicating 'Showing rows: 1 to 1' and 'Page No: 1 of 1'.

Рисунок 4 — Результаты Запроса 3

Запрос 4: Поиск самого частого клиента за прошлый год.

Цель: Найти клиента с максимальным количеством визитов за 2025 год.

Что я сделал: С помощью функции EXTRACT(YEAR ...) из даты заказа выделил год и наложил условие выборки только за 2025 год. Данные сгруппировал по клиентам с подсчетом количества договоров.



The screenshot shows a database query interface. The top section is titled "Запрос История запросов" (Query History). Below it, the SQL query is displayed in a text area with line numbers 34 to 47. The query is as follows:

```
34 SELECT client.full_name,
35
36
37
38
39 SELECT client.full_name AS "Клиент", COUNT(contract.contract_id) AS "Визитов в 2025 году"
40 FROM client
41 JOIN contract ON client.client_id = contract.client_id
42 WHERE EXTRACT(YEAR FROM contract.order_date) = 2025
43 GROUP BY client.full_name
44 ORDER BY "Визитов в 2025 году" DESC
45 LIMIT 1;
46
47
```

Below the query, there is a section titled "Результат" (Result). It shows a table with two columns: "Клиент" (Client) and "Визитов в 2025 году" (Visits in 2025). The table has one row of data:

	Клиент character varying (150)	Визитов в 2025 году bigint
1	Алексеев А.А.	2

The interface also includes a toolbar with various icons for query execution and a status bar at the bottom indicating "Showing rows: 1 to 1" and "Page No: 1 of 1".

Рисунок 5 — Результаты Запроса 4

Запрос 5: Расчет выручки мастеров за месяц.

Цель: Узнать суммарный заработок каждого механика за февраль 2026 года.

Что я сделал: Произвел фильтрацию завершенных работ с извлечением месяца и года из фактической даты. Стоимость выполненных работ просуммировал в разрезе каждого сотрудника.

The screenshot shows a database query interface. At the top, there are tabs for 'Запрос' (Query) and 'История запросов' (Query History). The main area displays the following SQL query:

```
45 LIMIT 1;  
46  
47  
48  
49  
50 SELECT employee.full_name AS "Механик", SUM(work_assignment.partial_work_cost) AS "Заработок за Февраль"  
51 FROM employee  
52 JOIN work_assignment ON employee.employee_id = work_assignment.employee_id  
53 WHERE EXTRACT(MONTH FROM work_assignment.actual_completion_date) = 2  
54 AND EXTRACT(YEAR FROM work_assignment.actual_completion_date) = 2026  
55 GROUP BY employee.full_name;  
56  
57  
58
```

Below the query editor, there are tabs for 'Результат' (Result), 'Сообщения' (Messages), and 'Уведомления' (Notifications). The 'Результат' tab is active, showing a table with two columns: 'Механик' (Mechanic) and 'Заработок за Февраль' (February Earnings). The table has two rows of data:

	Механик character varying (150)	Заработок за Февраль numeric
1	Косяков К.К.	5000.00
2	Универсалов У.У.	13000.00

At the bottom of the interface, there is a toolbar with various icons for file operations and a status bar showing 'Showing rows: 1 to 2', 'Page No: 1 of 1', and navigation buttons.

Рисунок 6 — Результата Запроса 5

Запрос 6: Поиск вернувшихся клиентов.

Цель: Собрать контактные данные автовладельцев, обращавшихся в мастерскую более одного раза.

Что я сделал: Сгруппировал данные по ID клиента. Поставил условие, что контрактов больше одного.

The screenshot shows a database query interface. At the top, there's a tab labeled "Запрос" (Query) and "История запросов" (Query History). Below the tab, a SQL query is entered in a text area:

```
56
57
58
59
60
61 SELECT client.full_name, client.phone, COUNT(contract.contract_id) AS "Кол-во обращений"
62 FROM client
63 JOIN contract ON client.client_id = contract.client_id
64 GROUP BY client.client_id, client.full_name, client.phone
65 HAVING COUNT(contract.contract_id) > 1;
66
67
68
69
```

Below the query, there's a section for the result. It includes tabs for "Результат" (Result), "Сообщения" (Messages), and "Уведомления" (Notifications). The "Результат" tab is active, showing a table with the following data:

	full_name character varying (150)	phone character varying (20)	Кол-во обращений bigint
1	Борисов Б.Б.	222	2
2	Алексеев А.А.	111	2

Below the table, there's a status bar showing "Showing rows: 1 to 2" and "Page No: 1 of 1".

Рисунок 7 — Результаты Запроса 6

Запрос 7: Расчет финансовых штрафов для механиков.

Цель: Рассчитать штраф, где сам штраф это 5% от стоимости работы за каждый день просрочки, для механиков за февраль 2026 года.

Что я сделал: Выбрал заказы, завершённые в указанном месяце с нарушением сроков. Внутри агрегатной функции реализовал математическую формулу: $(\text{Кол-во дней просрочки}) * 0.05 * (\text{Стоимость работы})$.

The screenshot shows a database query interface with a tab labeled "Запрос" (Query) and "История запросов" (Query History). The SQL query is as follows:

```
72
73
74 SELECT employee.full_name AS "Механик",
75        SUM((contract.actual_completion_date - contract.planned_completion_date) * 0.05 * work_assignment.)
76 FROM employee
77 JOIN work_assignment ON employee.employee_id = work_assignment.employee_id
78 JOIN contract_service ON work_assignment.contract_service_id = contract_service.contract_service_id
79 JOIN contract ON contract_service.contract_id = contract.contract_id
80 WHERE contract.actual_completion_date > contract.planned_completion_date
81        AND EXTRACT(MONTH FROM contract.actual_completion_date) = 2
82        AND EXTRACT(YEAR FROM contract.actual_completion_date) = 2026
83 GROUP BY employee.full_name;
84
85
```

Below the query editor, there are tabs for "Результат" (Result), "Сообщения" (Messages), and "Уведомления" (Notifications). The "Результат" tab is active, showing a table with the following data:

	Механик character varying (150)	Сумма штрафа numeric
1	Косяков К.К.	750.0000

The interface also includes a toolbar with icons for various actions and a status bar indicating "Showing rows: 1 to 1" and "Page No: 1 of 1".

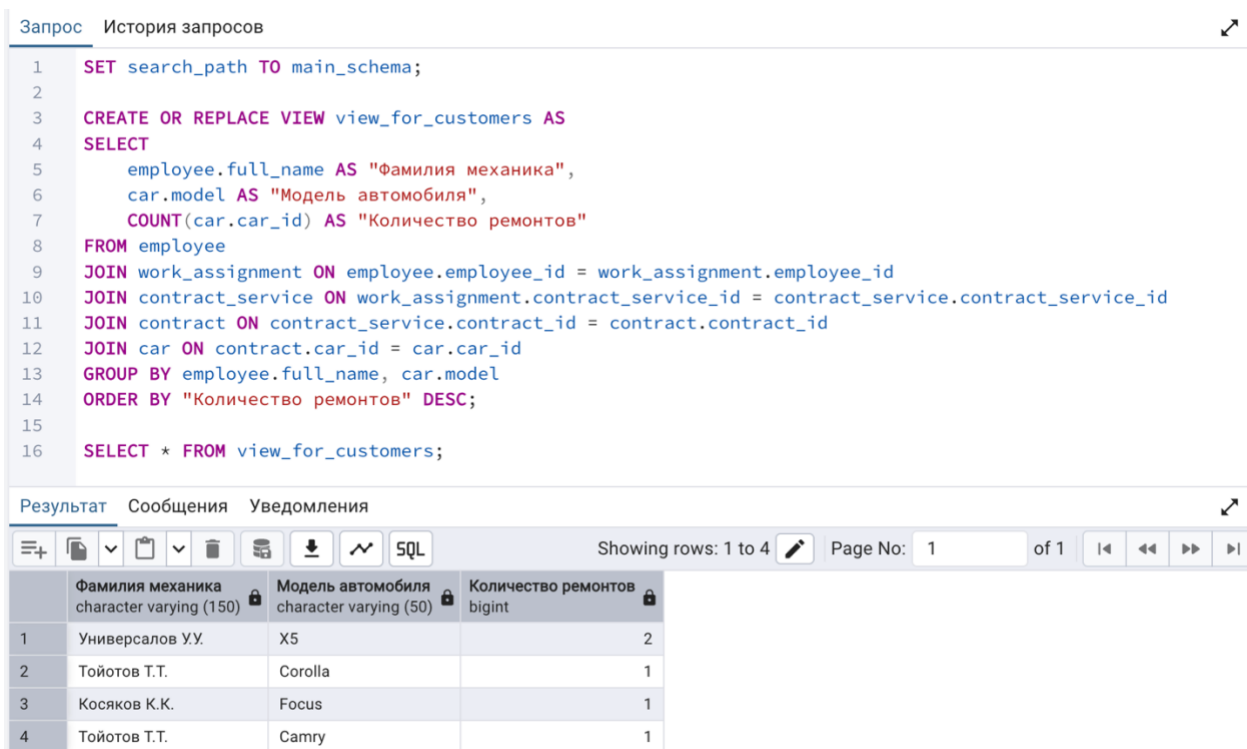
Рисунок 8 — Результаты Запроса 7

Создание виртуальных таблиц

Представление 1 (Для заказчиков): Статистика специализации мастеров.

Цель: Создать виртуальную таблицу, которая скрывает структуру БД от клиентского приложения и показывает только ФИО мастера и модель авто, которую он ремонтирует чаще всего.

Что я сделал: Написал запрос с объединением 5 таблиц. Конструкцию сохранил в БД с помощью команды CREATE VIEW.



Запрос История запросов

```
1 SET search_path TO main_schema;
2
3 CREATE OR REPLACE VIEW view_for_customers AS
4 SELECT
5     employee.full_name AS "Фамилия механика",
6     car.model AS "Модель автомобиля",
7     COUNT(car.car_id) AS "Количество ремонтов"
8 FROM employee
9 JOIN work_assignment ON employee.employee_id = work_assignment.employee_id
10 JOIN contract_service ON work_assignment.contract_service_id = contract_service.contract_service_id
11 JOIN contract ON contract_service.contract_id = contract.contract_id
12 JOIN car ON contract.car_id = car.car_id
13 GROUP BY employee.full_name, car.model
14 ORDER BY "Количество ремонтов" DESC;
15
16 SELECT * FROM view_for_customers;
```

Результат Сообщения Уведомления

Showing rows: 1 to 4 Page No: 1 of 1

	Фамилия механика character varying (150)	Модель автомобиля character varying (50)	Количество ремонтов bigint
1	Универсалов У.У.	X5	2
2	Тойотов Т.Т.	Corolla	1
3	Косяков К.К.	Focus	1
4	Тойотов Т.Т.	Camry	1

Рисунок 9 — Результат выборки из VIEW

Представление 2 (Для менеджеров): Автоматический расчет премий.

Цель: Сформировать динамический отчет для начисления премий механикам, закрывшим все февральские заказы без нарушения дедлайнов. Премия это 10% от заработка.

Что я сделал: Создал представление, отфильтровывающее просроченные заказы. Добавил математическое вычисление 10% от суммы выполненных работ.

Запрос

История запросов

1

SET search_path TO main_schema;

2

3

CREATE OR REPLACE VIEW view_for_managers AS

4

SELECT

5

employee.full_name AS "Фамилия механика",

6

SUM(work_assignment.partial_work_cost) * 0.10 AS "Премия"

7

FROM employee

8

JOIN work_assignment ON employee.employee_id = work_assignment.employee_id

9

JOIN contract_service ON work_assignment.contract_service_id = contract_service.contract_service_id

10

JOIN contract ON contract_service.contract_id = contract.contract_id

11

12

WHERE contract.actual_completion_date <= contract.planned_completion_date

13

AND EXTRACT(MONTH FROM contract.actual_completion_date) = 2

14

AND EXTRACT(YEAR FROM contract.actual_completion_date) = 2026

15

GROUP BY employee.full_name;

16

17

SELECT * FROM view_for_managers;

Результат

Сообщения

Уведомления

≡

📄

▼

📋

▼

🗑️

🔍

📥

📄

📄

SQL

Showing rows: 1 to 1

Page No: 1

of 1

⏪

⏩

⏴

⏵

	Фамилия механика character varying (150)	Премия numeric
1	Универсалов У.У.	1300.0000

Рисунок 10 — Результат выборки из VIEW

Модификация данных с использованием вложенных подзапросов

Команда UPDATE.

Цель: Реализовать программу лояльности: дать скидку 10% на ремонт всем автомобилям марки «Ford».

Что я сделал: Использовал команду UPDATE. В блоке условия WHERE применил вложенный подзапрос, который искал ID договоров, связанных с машинами нужной марки.

Запрос

История запросов

1

SET search_path TO main_schema;

2

3

SELECT

4

contract_service.contract_service_id,

5

car.brand AS "Бренд машины",

6

contract_service.work_cost AS "Стоимость работы"

7

FROM contract_service

8

JOIN contract ON contract_service.contract_id = contract.contract_id

9

JOIN car ON contract.car_id = car.car_id

10

ORDER BY contract_service.contract_service_id;

Результат

Сообщения

Уведомления

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	contract_service_id integer	Бренд машины character varying (50)	Стоимость работы numeric (10,2)
1	1	Toyota	10000.00
2	2	Toyota	3000.00
3	3	BMW	10000.00
4	4	BMW	3000.00
5	5	Ford	5000.00

Рисунок 11 — Результаты ДО использования команды

Запрос История запросов

```
1 SET search_path TO main_schema;  
2  
3 UPDATE contract_service  
4 SET work_cost = contract_service.work_cost * 0.90  
5 WHERE contract_service.contract_id IN (  
6     SELECT contract.contract_id  
7     FROM contract  
8     JOIN car ON contract.car_id = car.car_id  
9     WHERE car.brand = 'Ford'  
10 );
```

Результат Сообщения Уведомления

UPDATE 1

Запрос завершён успешно, время выполнения: 63 мс.

Рисунок 12 — Использование команды UPDATE

Запрос

История запросов

```

1  SET search_path TO main_schema;
2
3  SELECT
4      contract_service.contract_service_id,
5      car.brand AS "Бренд машины",
6      contract_service.work_cost AS "Стоимость работы"
7  FROM contract_service
8  JOIN contract ON contract_service.contract_id = contract.contract_id
9  JOIN car ON contract.car_id = car.car_id
10 ORDER BY contract_service.contract_service_id;

```

Результат

Сообщения

Уведомления

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	contract_service_id integer	Бренд машины character varying (50)	Стоимость работы numeric (10,2)
1	1	Toyota	10000.00
2	2	Toyota	3000.00
3	3	BMW	10000.00
4	4	BMW	3000.00
5	5	Ford	4500.00

Рисунок 13 — Результаты ПОСЛЕ использования команды

Команда INSERT.

- **Цель:** Запустить акцию, в которой автоматически добавляется бонусная услуга «Мойка» (со стоимостью 0) во все архивные договоры за 2025 год.

Что я сделал: Использовал конструкция INSERT INTO ... SELECT. Вложенные подзапросы динамически находили ID услуги "Мойка" в справочнике и привязывали ее ко всем найденным ID договоров 2025 года.

Запрос

История запросов

1

SET search_path TO main_schema;

2

3

SELECT

4

contract_service.contract_id AS "Номер договора (ID)",

5

EXTRACT(YEAR FROM contract.order_date) AS "Год заказа",

6

service.name AS "Услуга",

7

contract_service.work_cost AS "Стоимость"

8

FROM contract_service

9

JOIN contract ON contract_service.contract_id = contract.contract_id

10

JOIN service ON contract_service.service_id = service.service_id

11

ORDER BY contract_service.contract_id, service.name;

Результат

Сообщения

Уведомления

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	Номер договора (ID) integer	Год заказа numeric	Услуга character varying (150)	Стоимость numeric (10,2)
1	1	2025	Ремонт ДВС	10000.00
2	2	2025	ТО	3000.00
3	3	2026	Ремонт ДВС	10000.00
4	4	2026	ТО	3000.00
5	5	2026	Электрика	5000.00

Рисунок 14 — Результаты ДО использования команды

Запрос История запросов

```
1 SET search_path TO main_schema;
2
3 INSERT INTO contract_service (contract_id, service_id, work_cost)
4 SELECT
5     contract.contract_id,
6     (SELECT service.service_id FROM service WHERE service.name = 'Мойка'),
7     0
8 FROM contract
9 WHERE EXTRACT(YEAR FROM contract.order_date) = 2025;
```

Результат Сообщения Уведомления

INSERT 0 2

Запрос завершён успешно, время выполнения: 57 мс.

Рисунок 15 — Использование команды INSERT

Запрос

История запросов

```

1  SET search_path TO main_schema;
2
3  SELECT
4      contract_service.contract_id AS "Номер договора (ID)",
5      EXTRACT(YEAR FROM contract.order_date) AS "Год заказа",
6      service.name AS "Услуга",
7      contract_service.work_cost AS "Стоимость"
8  FROM contract_service
9  JOIN contract ON contract_service.contract_id = contract.contract_id
10 JOIN service ON contract_service.service_id = service.service_id
11 ORDER BY contract_service.contract_id, service.name;

```

Результат

Сообщения

Уведомления

+

📄

▼

📋

▼

🗑️

🗑️

📥

⬇️

📈

SQL

SQL

	Номер договора (ID) integer	Год заказа numeric	Услуга character varying (150)	Стоимость numeric (10,2)
1	1	2025	Мойка	0.00
2	1	2025	Ремонт ДВС	10000.00
3	2	2025	Мойка	0.00
4	2	2025	ТО	3000.00
5	3	2026	Ремонт ДВС	10000.00
6	4	2026	ТО	3000.00
7	5	2026	Электрика	5000.00

Рисунок 16 — Результаты ПОСЛЕ использования команды

Команда DELETE.

- **Цель:** Удалить из справочника автомобили, которые ни разу не обслуживались в сервисе.
- **Что я сделал:** Использовал оператор DELETE с условием NOT IN. Подзапрос формировал список всех автомобилей, имеющих договоры. Главный запрос удалял те машины, ID которых в этом списке не оказалось.
-

Запрос

История запросов

```
1 SET search_path TO main_schema;
2 INSERT INTO car (client_id, license_plate, brand, model, power, production_year)
3 VALUES (1, 'Y999YY', 'Lada', 'Vesta', 106, 2022);
4
5 SELECT car_id, brand, model, license_plate FROM car;
```

Результат

Сообщения

Уведомления

SQL

Showing rows: 1 to 5

Page No: 1

	car_id [PK] integer	brand character varying (50)	model character varying (50)	license_plate character varying (20)
1	1	Toyota	Camry	A123AA
2	2	Toyota	Corolla	B456BB
3	3	BMW	X5	C789CC
4	4	Ford	Focus	X000XX
5	5	Lada	Vesta	Y999YY

Рисунок 17 — Результаты ДО использования команды

Запрос История запросов

```
1 SET search_path TO main_schema;  
2  
3 DELETE FROM car  
4 WHERE car.car_id NOT IN (  
5     SELECT contract.car_id FROM contract  
6 );
```

Результат Сообщения Уведомления

DELETE 1

Запрос завершён успешно, время выполнения: 55 мс.

Рисунок 18 — Использование команды DELETE

Запрос

История запросов

1

`SELECT car_id, brand, model, license_plate FROM car;`

Результат

Сообщения

Уведомления

≡+

▼

▼

SQL

Showing rows: 1 to 4

	car_id [PK] integer	brand character varying (50)	model character varying (50)	license_plate character varying (20)
1	1	Toyota	Camry	A123AA
2	2	Toyota	Corolla	B456BB
3	3	BMW	X5	C789CC
4	4	Ford	Focus	X000XX

Рисунок 19 — Результаты ПОСЛЕ использования команды

Индексы и анализ планов запросов

Простой индекс.

Цель: Оценить влияние создания B-Tree индекса на скорость поиска конкретного клиента.

Для проведения эксперимента таблица client была искусственно расширена до 10 000 записей. Поиск клиента по номеру телефона осуществлялся с помощью команды EXPLAIN ANALYZE.

1. **Без индекса:** Оптимизатор СУБД применил метод полного последовательного сканирования таблицы (Seq Scan). Для нахождения одной записи базе данных потребовалось прочитать и отфильтровать более 10 000 строк (Rows Removed by Filter: 10002). Время выполнения составило 0.691 мс.
2. **С индексом:** После создания простого B-Tree индекса на колонку phone план запроса кардинально изменился. Оптимизатор применил метод индексного сканирования (Index Scan). База данных обратилась напрямую к нужному блоку памяти, минуя перебор лишних строк. Время выполнения снизилось до 0.049 мс.

Запрос

История запросов

```

1  SET search_path TO main_schema;
2
3
4  INSERT INTO client (full_name, phone, email)
5  SELECT
6      'Фейковый клиент' || i,
7      '8000000' || lpad(i::text, 5, '0'),
8      'fake' || i || '@test.ru'
9  FROM generate_series(1, 10000) AS i;
10
11 ANALYZE client;
12
13 EXPLAIN ANALYZE SELECT * FROM client WHERE phone = '800000005000';

```

Результат

Сообщения

Уведомления

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

Showing rows: 1 to 5 

Page 1

	QUERY PLAN
	text 
1	Seq Scan on client (cost=0.00..249.04 rows=1 width=580) (actual time=0.354..0.683 rows=1 loop...
2	Filter: ((phone)::text = '800000005000'::text)
3	Rows Removed by Filter: 10002
4	Planning Time: 0.100 ms
5	Execution Time: 0.691 ms

Рисунок 20 — Результаты поиска поиска ДО индекса

Запрос

История запросов

1

SET search_path TO main_schema;

2

3

CREATE INDEX idx_client_phone ON client(phone);

4

5

EXPLAIN ANALYZE SELECT * FROM client WHERE phone = '80000005000';

Результат

Сообщения

Уведомления

≡+

SQL

Showing rows: 1 to 4

Page No: 1

	QUERY PLAN
	text
1	Index Scan using idx_client_phone on client (cost=0.29..8.30 rows=1 width=580) (actual time=0.032..0.033 rows=1 loo...
2	Index Cond: ((phone)::text = '80000005000'::text)
3	Planning Time: 0.382 ms
4	Execution Time: 0.049 ms

Рисунок 21 —Создания индекса и новые результаты

Запрос История запросов

1 DROP INDEX main_schema.idx_client_phone;

Результат Сообщения Уведомления

DROP INDEX

Запрос завершён успешно, время выполнения: 55 мс.

Рисунок 22 — Удаление индекса

Составной индекс

Цель: Доказать эффективность индексов при поиске по нескольким критериям (марка авто и год выпуска) на реалистичном объеме данных.

Для тестирования составного индекса был написан скрипт генерации данных, который увеличил объем таблицы car до 50 000 уникальных записей. Искомый запрос включал фильтрацию сразу по двум столбцам: марка и год выпуска. Под заданные условия попадает около 30 000 строк.

1. **Без индекса:** При отсутствии индекса оптимизатор был вынужден применить метод полного сканирования (Seq Scan). База данных прочитала все 50 000 строк таблицы и последовательно отфильтровала их по двум условиям. Время выполнения 3.708 мс.
2. **С составным индексом:** После создания составного индекса, оптимизатор применил связку методов Bitmap Index Scan и Bitmap Heap Scan. Сначала база данных обратилась к индексу, чтобы мгновенно составить битовую карту (физического расположения нужных 30 000 строк. Затем по этой карте данные были извлечены из таблицы. Время выполнения сократилось до 2.859мс.

Запрос

История запросов

```
1 SET search_path TO main_schema;
2
3 INSERT INTO car (client_id, license_plate, brand, model, power, production_year)
4 SELECT
5     1,
6     'Ф' || lpad(i::text, 7, '0') || 'ФФ',
7     CASE WHEN i % 2 = 0 THEN 'Toyota' ELSE 'Nissan' END,
8     'Model ' || i,
9     100,
10    2020
11 FROM generate_series(1, 50000) AS i;
12
13 ANALYZE car;
14
15 EXPLAIN ANALYZE SELECT * FROM car WHERE brand = 'Toyota' AND production_year = 2020;
```

Результат

Сообщения

Уведомления

Showing rows: 1 to 7

Page No: 1 of 1

	QUERY PLAN
1	Bitmap Heap Scan on car (cost=410.06..1518.52 rows=29831 width=242) (actual time=0.993..2.589 rows=30001 loops=1)
2	Recheck Cond: (((brand)::text = 'Toyota'::text) AND (production_year = 2020))
3	Heap Blocks: exact=661
4	-> Bitmap Index Scan on idx_car_brand_year (cost=0.00..402.60 rows=29831 width=0) (actual time=0.937..0.937 rows=30001 loops=1)
5	Index Cond: (((brand)::text = 'Toyota'::text) AND (production_year = 2020))
6	Planning Time: 0.184 ms
7	Execution Time: 3.708 ms

Рисунок 22 — Результаты поиска поиска ДО индекса

Запрос История запросов	
1	SET search_path TO main_schema;
2	
3	CREATE INDEX idx_car_brand_year ON car (brand, production_year);
4	
5	EXPLAIN ANALYZE SELECT * FROM car WHERE brand = 'Toyota' AND production_year = 2020;
Результат Сообщения Уведомления	
≡ 📄 ▼ 📋 ▼ 🗑️ 📦 ⬇️ 📈 SQL Showing rows: 1 to 7 Page No: 1 of	
	QUERY PLAN text 🔒
1	Bitmap Heap Scan on car (cost=420.44..1532.38 rows=30063 width=243) (actual time=0.499..2.282 rows=30001 loops=1)
2	Recheck Cond: (((brand)::text = 'Toyota'::text) AND (production_year = 2020))
3	Heap Blocks: exact=661
4	-> Bitmap Index Scan on idx_car_brand_year (cost=0.00..412.92 rows=30063 width=0) (actual time=0.451..0.451 rows=30001 loops=1)
5	Index Cond: (((brand)::text = 'Toyota'::text) AND (production_year = 2020))
6	Planning Time: 0.111 ms
7	Execution Time: 2.859 ms

Рисунок 22 —Создания индекса и новые результаты

Запрос История запросов	
1	SET search_path TO main_schema;
2	
3	CREATE INDEX idx_car_brand_year ON car (brand, production_year);
4	
5	EXPLAIN ANALYZE SELECT * FROM car WHERE brand = 'Toyota' AND production_year = 2020;
Результат Сообщения Уведомления	
Запрос выполнен успешно. Общее время выполнения: 169 мс. обработано строк: 7.	

Рисунок 23—Удаление индекса

ВЫВОДЫ

В ходе выполнения лабораторной работы были получены практические навыки работы с данными в СУБД PostgreSQL. Освоено применение вложенных подзапросов, выступающих в роли динамических параметров для операций INSERT, UPDATE и DELETE.

Особое внимание было уделено представлениям, которые позволяют создавать удобные виртуальные отчеты для конечных пользователей.

На практике изучен инструмент планировщика запросов EXPLAIN ANALYZE. Опытным путем, с использованием генерации доп значений, доказана эффективность построения составных индексов.